

Strongly Connected Components

A strongly connected component (SCC) of a directed graph is a maximal subset of vertices such that there is a path from every vertex to every other vertex in the subset.

Natural Explanation

In a directed graph, an SCC is a group of vertices where you can go from any vertex to any other vertex by following directed edges.

Think of cities connected by one-way streets: an SCC is a set of cities where you can drive from any city to any other city in the set.

Formal Definition

A subset $S \subseteq V$ is strongly connected if: - For every pair of distinct vertices $u, v \in S$, there exists a path from u to v and a path from v to u

SCC partition: partition all vertices into maximal strongly connected components (no vertex appears in multiple SCCs; union covers all vertices).

Algorithm: Kosaraju's Algorithm

Two-pass DFS approach:

KOSARAJU(graph)

```
// Pass 1: DFS on original graph to get finish times
finish_order = []
visited = empty set
for each vertex v:
    if v not in visited:
        DFS-VISIT(v, visited, finish_order)

// Pass 2: DFS on transpose graph in reverse finish order
transpose = reverse all edge directions in graph
visited = empty set
scc_list = []
for each vertex v in reverse(finish_order):
    if v not in visited:
        scc = DFS-VISIT-COMPONENT(v, transpose, visited)
        scc_list.append(scc)
```

```
return scc_list
```

Time: $O(|V| + |E|)$ — two DFS passes

Intuition: first DFS discovers finish times; reversing the graph and processing in that order isolates SCCs.

Algorithm: Tarjan's Algorithm

Single-pass approach using a stack:

Maintain a stack of vertices and track discovery times and low values: - **discovery**[v] = when v is first visited - **low**[v] = earliest discovery time reachable from v (including back edges)

When **low**[v] == **discovery**[v], v is the root of an SCC. Pop the stack to extract that SCC.

Time: $O(|V| + |E|)$ — single DFS

Advantage: single pass instead of two passes; often faster in practice.

Properties

Acyclic condensation: if you collapse each SCC to a single supernode, the resulting graph is a DAG (directed acyclic graph).

Unique partition: every vertex belongs to exactly one SCC.

Use Cases

- **Deadlock detection:** in resource allocation, SCCs represent potential cyclic dependencies
 - **Compiler optimization:** for loop analysis and instruction scheduling
 - **Social networks:** finding tightly-knit groups (e.g., communities that cite each other)
 - **Circuit design:** finding combinatorial loops
 - **Software testing:** finding strongly dependent modules
-

Related

- [[Depth-First Search]]
- [[Graphs]]

- [[Topological Sorting]]